

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG
BỘ MÔN MÁY TÍNH VÀ HỆ THỐNG NHÚNG

BÁO CÁO ĐỒ ÁN MÔN HỌC

Học phần: THỰC HÀNH HỆ THỐNG NHÚNG

Lớp: 22DTV_CLC — Nhóm đăng ký: 17

Đề tài số 13

**SOC – ETHERNET TÍCH HỢP HỆ THỐNG,
FIRMWARE VÀ ỨNG DỤNG NHẬN LỆNH TCP TỪ
PC**

Giảng viên phụ trách : ThS. TRẦN TUẤN KIỆT
ThS. ĐỖ QUỐC MINH ĐĂNG
ThS. NGUYỄN NHƯ HOÀNG

Thành viên 1 : Văn Đình Nam — 22207063
Thành viên 2 : Lương Hải Long — 22207056
Thành viên 3 : Trần Sĩ Nam — 22207062
Thành viên 4 : Lê Tấn Phi Pha — 22207066
Thành viên 5 : Vũ Châu Thắng Lợi — 22207055

Thành phố Hồ Chí Minh – Tháng 4 năm 2026

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG
BỘ MÔN MÁY TÍNH VÀ HỆ THỐNG NHÚNG
BÁO CÁO ĐỒ ÁN MÔN HỌC

Học phần: THỰC HÀNH HỆ THỐNG NHÚNG — 22DTV_CLC

Đề tài số 13 — Nhóm đăng ký: 17

SOC – ETHERNET TÍCH HỢP HỆ THỐNG, FIRMWARE
VÀ ỨNG DỤNG NHẬN LỆNH TCP TỪ PC

SoC-Ethernet Integrated System, Firmware, and PC Application for TCP Command Reception

Giảng viên phụ trách : ThS. TRẦN TUẤN KIỆT
ThS. ĐỖ QUỐC MINH ĐĂNG
ThS. NGUYỄN NHƯ HOÀNG

Nền tảng phần cứng : DE10-Standard Cyclone V SoC FPGA
Hướng hiện thực : HPS Linux nhận lệnh TCP, chuẩn hóa payload và ghi kết quả
xuống cụm HEX qua bridge HPS-FPGA
Trọng tâm báo cáo : Giao tiếp Ethernet/TCP từ PC tới board

Thành viên 1 : Văn Đình Nam — 22207063
Thành viên 2 : Lương Hải Long — 22207056
Thành viên 3 : Trần Sĩ Nam — 22207062
Thành viên 4 : Lê Tấn Phi Pha — 22207066
Thành viên 5 : Vũ Châu Thắng Lợi — 22207055

BẢNG PHÂN CÔNG CÔNG VIỆC VÀ CHẤM ĐIỂM CHÉO

STT	Thành viên	MSSV	Công việc phụ trách	Điểm chấm chéo
1	Lương Hải Long	22207056	Làm slide; biên soạn LaTeX/Typst; phát triển firmware Python phía SoC (luồng điều khiển chính, giao tiếp với FPGA qua memory-mapped I/O, xử lý dữ liệu cục bộ); thực hiện phần Quartus; cài hệ điều hành Linux cho microSD; tìm hiểu và đề xuất hướng đi đề tài.	10/10
2	Lê Tấn Phi Pha	22207066	Phát triển firmware Python phía truyền thông (đóng gói/giải mã dữ liệu, dịch vụ TCP qua Ethernet, liên kết với app Android); phát triển app Android; thực hiện phần Quartus; cấu hình nạp Ethernet từ Linux qua router; cấu hình Verilog.	10/10
3	Văn Đình Nam	22207063	Tích hợp các IP trong Quartus cho hệ thống FPGA; biên dịch và nạp file .sof xuống FPGA; đồng bộ phần Verilog với firmware; kiểm định chất lượng đồ án.	9/10
4	Trần Sĩ Nam	22207062	Đồng thực hiện PowerPoint slide; đồng kiểm thử và hiệu đính; tester firmware Python và app Android.	9/10
5	Vũ Châu Thắng Lợi	22207055	Góp ý hướng đi cho cả nhóm; hỗ trợ nạp FPGA và kiểm thử chất lượng đồ án.	9/10

LỜI CẢM ƠN

Trước hết, nhóm thực hiện xin trân trọng cảm ơn Ban Giám hiệu, Ban Chủ nhiệm Khoa Điện tử – Viễn thông và Bộ môn Máy tính và Hệ thống nhúng, Trường Đại học Khoa học Tự nhiên, Đại học Quốc gia Thành phố Hồ Chí Minh, đã xây dựng chương trình đào tạo nền tảng về thiết kế số, hệ điều hành nhúng và SoC FPGA. Hệ thống kiến thức đó là cơ sở để nhóm tiếp cận đề tài **SoC – Ethernet tích hợp hệ thống, firmware và ứng dụng nhận lệnh TCP từ PC** như một bài toán tích hợp phần cứng – phần mềm có thể kiểm chứng được trong phạm vi học phần.

Nhóm xin chân thành cảm ơn quý Thầy phụ trách học phần **THỰC HÀNH HỆ THỐNG NHÚNG** — ThS. TRẦN TUẤN KIẾT, ThS. ĐỖ QUỐC MINH ĐĂNG và ThS. NGUYỄN NHƯ HOÀNG — đã đồng hành trong suốt quá trình triển khai. Sự định hướng kịp thời từ quý Thầy đã giúp nhóm khoanh vùng phạm vi phù hợp với năng lực thực hành, đồng thời giữ được mạch kỹ thuật xuyên suốt từ khâu cấu hình phần cứng tới khâu kiểm thử đầu cuối.

Nhóm xin gửi lời cảm ơn đặc biệt đến ThS. ĐỖ QUỐC MINH ĐĂNG vì những trao đổi chuyên môn rất thiết thực ở giai đoạn bring-up. Việc xác định rõ vai trò của **USB-UART** như console hệ thống của Linux trên DE10-Standard, phân biệt với **JTAG-UART**, cũng như gợi ý hai con đường truy cập board (qua cổng COM USB-UART hoặc qua SSH trong cùng mạng LAN) đã giúp nhóm giải quyết được nút thắt khó đoán nhất ở giai đoạn đầu. Nhờ đó, hệ thống mới bước qua được tầng **Linux shell** → **IP hợp lệ** → **dịch vụ TCP** để tiến tới kết quả cuối cùng quan sát được trên cụm HEX.

Cuối cùng, nhóm xin cảm ơn các thành viên trong tập thể lớp **22DTV_CLC** đã chia sẻ kinh nghiệm, hỗ trợ thiết bị và đóng góp ý kiến cho bản báo cáo. Báo cáo chắc chắn còn nhiều thiếu sót; nhóm rất mong tiếp tục nhận được nhận xét, góp ý từ quý Thầy/Cô và bạn đọc để có thể hoàn thiện hơn trong những công trình kế tiếp.

Thành phố Hồ Chí Minh, Tháng 4 năm 2026
Nhóm sinh viên thực hiện

LỜI CAM ĐOAN

Nhóm thực hiện xin cam đoan rằng nội dung báo cáo đồ án môn học **THỰC HÀNH HỆ THỐNG NHÚNG — đề tài SoC – Ethernet tích hợp hệ thống, firmware và ứng dụng nhận lệnh TCP từ PC** — là kết quả nghiên cứu, triển khai và kiểm thử của chính nhóm trong khuôn khổ học phần. Toàn bộ phần cứng, phần mềm, hình ảnh thực nghiệm và đoạn mã nguồn được trình bày trong báo cáo phản ánh trung thực sản phẩm thực tế đã được kiểm chứng trên board DE10-Standard tại thời điểm hoàn thành đồ án.

Các hiện vật kỹ thuật được trình bày trong báo cáo, bao gồm cấu hình Platform Designer, mã nguồn HDL, dịch vụ TCP trên HPS Linux, ứng dụng PC Python/Tkinter và Android client, đều có thể đối chiếu trực tiếp với mã nguồn và kết quả vận hành thực tế. Những phân lý thuyết được tham khảo từ tài liệu chuyên ngành đã được trích dẫn đầy đủ ở phần **Tài liệu tham khảo**, theo chuẩn IEEE và được quản lý qua tệp BibTeX.

Nhóm cam đoan không sao chép công trình của tác giả khác mà không kèm trích dẫn nguồn, không sử dụng dữ liệu hoặc kết quả không được phép, và không gian dối trong việc mô tả mức độ hoàn thành của hệ thống. Cấu trúc, cách trình bày và quy ước trích dẫn của báo cáo được thực hiện theo quy định hiện hành của Khoa Điện tử – Viễn thông, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM [1].

Nếu phát hiện bất kỳ sai phạm nào liên quan đến tính trung thực học thuật, nhóm xin hoàn toàn chịu trách nhiệm theo quy định của Nhà trường.

Thành phố Hồ Chí Minh, Tháng 4 năm 2026

Đại diện nhóm thực hiện

Lương Hải Long

TÓM TẮT

Tên đề tài: SoC – Ethernet tích hợp hệ thống, firmware và ứng dụng nhận lệnh TCP từ PC.

Báo cáo trình bày quá trình thiết kế, triển khai và kiểm thử một nguyên mẫu **SoC – Ethernet** trên board DE10-Standard sử dụng vi mạch Cyclone V SoC FPGA. Hệ thống được tổ chức xoay quanh đường điều khiển **TCP-first**: phần Hard Processor System (HPS) chạy hệ điều hành Linux nhúng để khởi động, cấu hình mạng và phục vụ dịch vụ nhận lệnh; phần FPGA cung cấp một tập ngoại vi PIO cho cụm hiển thị 7 đoạn HEX0–HEX5, được ánh xạ tới không gian địa chỉ của HPS thông qua **Lightweight HPS-FPGA Bridge** [2], [3].

Trên board, dịch vụ `board_tcp_hex_server.py` lắng nghe ở cổng `5000/tcp`, nhận chuỗi ký tự gửi từ phía máy điều khiển, chuẩn hóa độ dài và tập ký tự, ánh xạ sang mã 7 đoạn (active-low) rồi ghi xuống các thanh ghi `0xFF200040 – 0xFF200090` bằng tiện ích `devmem`. Phía đầu cuối, ứng dụng PC viết bằng Python/Tkinter giữ vai trò chính: cho phép cấu hình IP/cổng, gõ chuỗi sáu ký tự, gửi tới board và ghi log phản hồi. Một ứng dụng Android client được xây dựng song song để đối chiếu hành vi của cùng một dịch vụ TCP với một đầu cuối khác trong cùng mạng cục bộ.

Kiểm thử đầu cuối xác nhận được chuỗi vận hành ổn định gồm năm bước: HPS boot Linux thành công, `eth0` có IP hợp lệ, server TCP lắng nghe đúng cổng, client gửi payload nhận về phản hồi `OK / ERR`, và cụm HEX hiển thị đúng chuỗi đã chuẩn hóa. Việc tách biệt giữa server TCP, lớp ánh xạ ký tự và lớp ghi thanh ghi cho phép khoanh vùng lỗi theo từng tầng, là một đóng góp về phương pháp triển khai trong phạm vi học phần.

Từ khóa: SoC FPGA, DE10-Standard, Cyclone V, HPS Linux, Lightweight HPS-FPGA Bridge, Ethernet, TCP socket, Python/Tkinter GUI, Android client, hiển thị 7 đoạn.

ABSTRACT

English title: SoC-Ethernet Integrated System, Firmware, and PC Application for TCP Command Reception.

This report describes the design, implementation, and verification of a prototype **SoC-Ethernet** system built on the Terasic DE10-Standard development board, which is based on the Intel Cyclone V SoC FPGA. The system is organized around a **TCP-first** control path. The Hard Processor System (HPS) runs an embedded Linux distribution that handles boot, network configuration, and the command-reception service, while the FPGA fabric provides a set of memory-mapped Parallel I/O peripherals driving the six-digit HEX0–HEX5 seven-segment display. These peripherals are exposed to the HPS through the **Lightweight HPS-FPGA Bridge** [2], [3].

On the board, the user-space service `board_tcp_hex_server.py` listens on port `5000/tcp`, accepts an incoming character string, sanitises both its length and its character set, encodes each character into the corresponding active-low seven-segment value, and writes the result to registers `0xFF200040` – `0xFF200090` via the `devmem` utility. On the host side, a Python/Tkinter desktop application is the primary endpoint: it allows the operator to configure the board IP/port, enter a six-character payload, dispatch the request, and inspect the textual `OK` / `ERR` reply. A complementary Android client is provided to verify that the same TCP service behaves consistently when accessed from a different endpoint inside the same local-area network.

Functional testing confirms a repeatable five-step workflow: successful Linux boot on the HPS, valid IP assignment on `eth0`, TCP server start-up on the configured port, end-to-end payload delivery with a textual reply, and correct rendering of the sanitised string on the HEX display. Decoupling the TCP server, the character-encoding layer, and the register-write layer enables disciplined per-layer fault isolation; this layering is presented as a methodological contribution within the scope of the practical course.

Keywords: SoC FPGA, DE10-Standard, Cyclone V, embedded Linux, Lightweight HPS-FPGA bridge, Ethernet, TCP socket, Python/Tkinter GUI, Android client, seven-segment display.

MỤC LỤC

Bảng phân công công việc	i
Lời cảm ơn	ii
Lời cam đoan	iii
Tóm tắt	iv
Abstract	v
Mục lục	vi
Danh mục chữ viết tắt	viii
Danh sách hình	x
1 Giới thiệu đề tài	1
1.1 Bối cảnh và động lực	1
1.2 Mục tiêu và phạm vi	1
1.3 Đóng góp và bố cục	2
2 Cơ sở hệ thống và định hướng thiết kế	3
2.1 Nền tảng Cyclone V SoC và DE10-Standard	3
2.2 HPS Linux và Bridge HPS-FPGA	4
2.3 Ethernet và giao thức TCP	5
2.4 Hiện thị 7 đoạn và mã hóa active-low	5
2.5 Công cụ phát triển	5
3 Thiết kế và triển khai hệ thống	6
3.1 Tổng quan kiến trúc	6
3.2 Thiết kế phần cứng	6
3.3 Triển khai phần mềm trên HPS	7
3.4 Ứng dụng đầu cuối	8
3.5 Giao thức ứng dụng	9
4 Kiểm thử và đánh giá hệ thống	10
4.1 Tiêu chí và môi trường	10
4.2 Kịch bản và kết quả	10
4.3 Phân tích lỗi và đánh giá	12
5 Kết luận và hướng phát triển	14

5.1 Kết quả và ý nghĩa	14
5.2 Hạn chế và hướng phát triển	14
Tài liệu tham khảo	16
Phụ lục	18

DANH MỤC CHỮ VIẾT TẮT

Chữ viết tắt	Diễn giải
API	Application Programming Interface — giao diện lập trình ứng dụng
APK	Android Package Kit — gói cài đặt ứng dụng Android
ARM	Advanced RISC Machines — kiến trúc bộ xử lý dùng cho HPS
DDR3	Double Data Rate Type 3 — chuẩn bộ nhớ động đồng bộ
DE10	DE10-Standard Development Kit (Terasic)
DHCP	Dynamic Host Configuration Protocol — RFC 2131
EMAC	Ethernet Media Access Controller (MAC trong HPS)
FPGA	Field-Programmable Gate Array
GUI	Graphical User Interface — giao diện đồ họa người dùng
HEX	Cụm LED 7 đoạn HEX0–HEX5 trên board DE10-Standard
HDL	Hardware Description Language — ngôn ngữ mô tả phần cứng
HPS	Hard Processor System — phần CPU cứng trong Cyclone V SoC
I/O	Input/Output
IDE	Integrated Development Environment
IEEE	Institute of Electrical and Electronics Engineers
IP	Internet Protocol
JTAG	Joint Test Action Group — chuẩn IEEE 1149.1
LAN	Local Area Network
MAC	Media Access Control
MicroSD	Micro Secure Digital — thẻ nhớ dùng cho boot Linux
MMIO	Memory-Mapped Input/Output
PIO	Parallel Input/Output (IP Avalon-MM của Intel)
Qsys	Tên cũ của Platform Designer (Intel)
RFC	Request for Comments
RGMII	Reduced Gigabit Media-Independent Interface
SoC	System-on-Chip

Chữ viết tắt	Diễn giải
SSH	Secure Shell — giao thức truy cập từ xa
TCP	Transmission Control Protocol — RFC 9293
TRM	Technical Reference Manual
UART	Universal Asynchronous Receiver/Transmitter
UDP	User Datagram Protocol
USB	Universal Serial Bus
VHDL	Very High Speed Integrated Circuit Hardware Description Language

DANH SÁCH HÌNH

Hình 1	Môi trường vận hành thực tế: PC điều khiển kết nối board DE10-Standard qua mạng cục bộ và kết quả hiển thị trên cụm HEX sau khi nhận lệnh TCP.	1
Hình 2	Trục TCP-first xuyên suốt báo cáo, từ ứng dụng người dùng tới ngoại vi vật lý. .	2
Hình 3	Các thành phần nền của giai đoạn bring-up: board, sơ đồ kết nối, công tắc MSEL và thẻ MicroSD chứa preloader, U-Boot và kernel.	4
Hình 4	Bản đồ địa chỉ được dùng nhất quán giữa Platform Designer, <code>hex_text.sh</code> và <code>board_tcp_hex_server.py</code>	6
Hình 5	Mã 7 đoạn của tập ký tự được hỗ trợ.	7
Hình 6	Giao diện ứng dụng PC và tình huống vận hành thực tế.	8
Hình 7	Môi trường kiểm thử thực tế: laptop, điện thoại và board kết nối qua cùng router; mọi đầu cuối đều có thể nhìn thấy <code>eth0</code> của board.	10
Hình 8	Kết quả với PC GUI (trái) và Android client (phải) trên cùng dịch vụ TCP. . . .	12
Hình 9	Sơ đồ khoanh vùng lỗi xếp theo thứ tự lan truyền vấn đề.	12

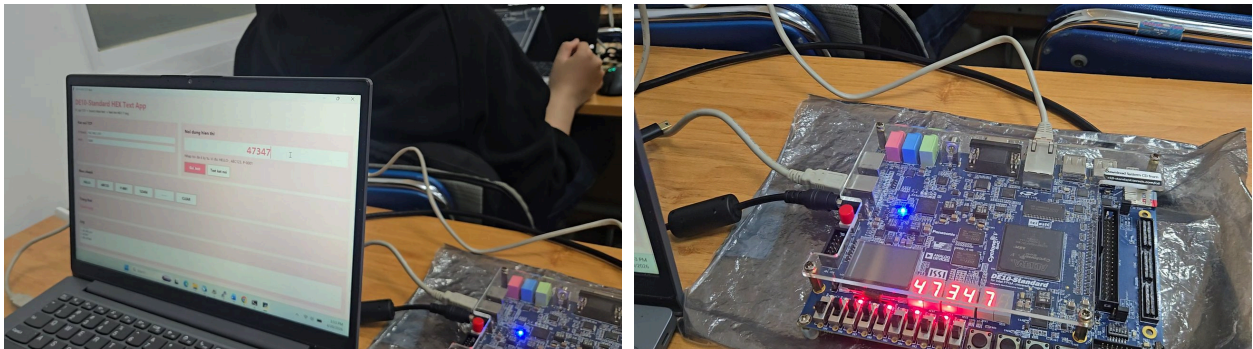
CHƯƠNG 1

Giới thiệu đề tài

1.1 Bối cảnh và động lực

Hệ thống nhúng hiện đại thường đòi hỏi đồng thời năng lực tính toán cho logic điều khiển và giao tiếp mạng cho cấu hình, giám sát từ xa. Kiến trúc **System-on-Chip** tích hợp Hard Processor System (HPS) cùng vùng vãi FPGA trên cùng một vi mạch là hướng đáp ứng tiêu biểu, trong đó HPS đảm nhận phần mềm hệ thống và FPGA cung cấp ngoại vi tùy biến độ trễ thấp [4], [5]. Board **DE10-Standard** (Terasic) trên chip **Cyclone V SoC FPGA** được sử dụng trong học phần **Thực hành Hệ thống nhúng** tại Khoa Điện tử – Viễn thông, Trường Đại học Khoa học Tự nhiên (ĐHQG-HCM) cho phép sinh viên đồng thời làm việc với HPS chạy Linux nhúng và một tập ngoại vi quen thuộc trên FPGA fabric [3], [6], [7].

Trong phạm vi đồ án, nhóm chủ động thu hẹp đề tài theo nhánh **TCP-first**: PC gửi chuỗi ký tự ngắn qua TCP, board nhận, phản hồi **OK / ERR** và hiển thị chuỗi đã chuẩn hóa trên cụm HEX. Việc thu hẹp giúp đề tài có hiện vật ở mọi tầng, dễ phân tách lỗi và dễ chứng minh tính đúng chức năng [8], [9].



Hình 1 — Môi trường vận hành thực tế: PC điều khiển kết nối board DE10-Standard qua mạng cục bộ và kết quả hiển thị trên cụm HEX sau khi nhận lệnh TCP.

1.2 Mục tiêu và phạm vi

Mục tiêu tổng quát là xây dựng nguyên mẫu SoC – Ethernet hoàn chỉnh, nhận lệnh từ PC qua TCP và phản hồi bằng cụm hiển thị 7 đoạn trên DE10-Standard. Các yêu cầu cụ thể:

- **Phần cứng:** hệ thống Platform Designer với HPS đầy đủ EMAC1/DDR3/SDIO/UART và sáu PIO 7 bit cho **HEX0 – HEX5** ; tổng hợp sạch trên Quartus Prime.

- **Hệ điều hành:** boot Linux nhúng từ MicroSD, vào shell qua USB-UART, kích hoạt `eth0` và xác nhận IP hợp lệ.
- **Dịch vụ board-side:** TCP server user-space lắng nghe `5000/tcp`, xử lý theo cấu trúc yêu cầu – phản hồi.
- **Lớp ánh xạ:** chuẩn hóa và mã hóa chuỗi sang mã 7 đoạn (active-low), ghi xuống các thanh ghi PIO trên **Lightweight HPS-FPGA Bridge**.
- **Đầu cuối:** GUI PC Python/Tkinter làm đầu cuối chính, Android client để đối chiếu hành vi.

Phạm vi giới hạn ở nguyên mẫu mạng cục bộ. Báo cáo không bao trùm đo hiệu năng định lượng, bảo mật mạng, đăng nhập tài khoản, lưu trữ cloud hoặc cập nhật từ xa.

1.3 Đóng góp và bố cục

Đóng góp chính nằm ở việc **tích hợp có hệ thống**: mở rộng boilerplate SoC-FPGA của Kashani [5] bằng cụm sáu PIO 7 bit cho HEX và đấu nối top-level VHDL; xây dựng dịch vụ TCP user-space kèm lớp chuẩn hóa ký tự và lớp ghi `devmem` trực tiếp; phát triển GUI PC và Android client trên cùng giao thức; đề xuất quy trình kiểm thử khoanh vùng lỗi theo tầng phù hợp với hệ SoC + Linux + ứng dụng người dùng.

Trục kỹ thuật xuyên suốt báo cáo

PC Python GUI → **TCP socket** → **HPS Linux server** → `seg()` + `devmem` → **HPS-FPGA Bridge** → **HEX0..HEX5**. Android client là đầu cuối đối chiếu trên cùng giao thức; UART-USB và SSH chỉ phục vụ bring-up.

Hình 2 — Trục TCP-first xuyên suốt báo cáo, từ ứng dụng người dùng tới ngoại vi vật lý.

Bố cục báo cáo gồm năm chương, không kể phần mở đầu và phụ lục: Chương 1 — Giới thiệu; Chương 2 — Cơ sở hệ thống và định hướng thiết kế; Chương 3 — Thiết kế và triển khai; Chương 4 — Kiểm thử và đánh giá; Chương 5 — Kết luận và hướng phát triển. Phần **Phụ lục** cung cấp mã nguồn cốt lõi và quy trình vận hành nhanh để tái lập hệ thống.

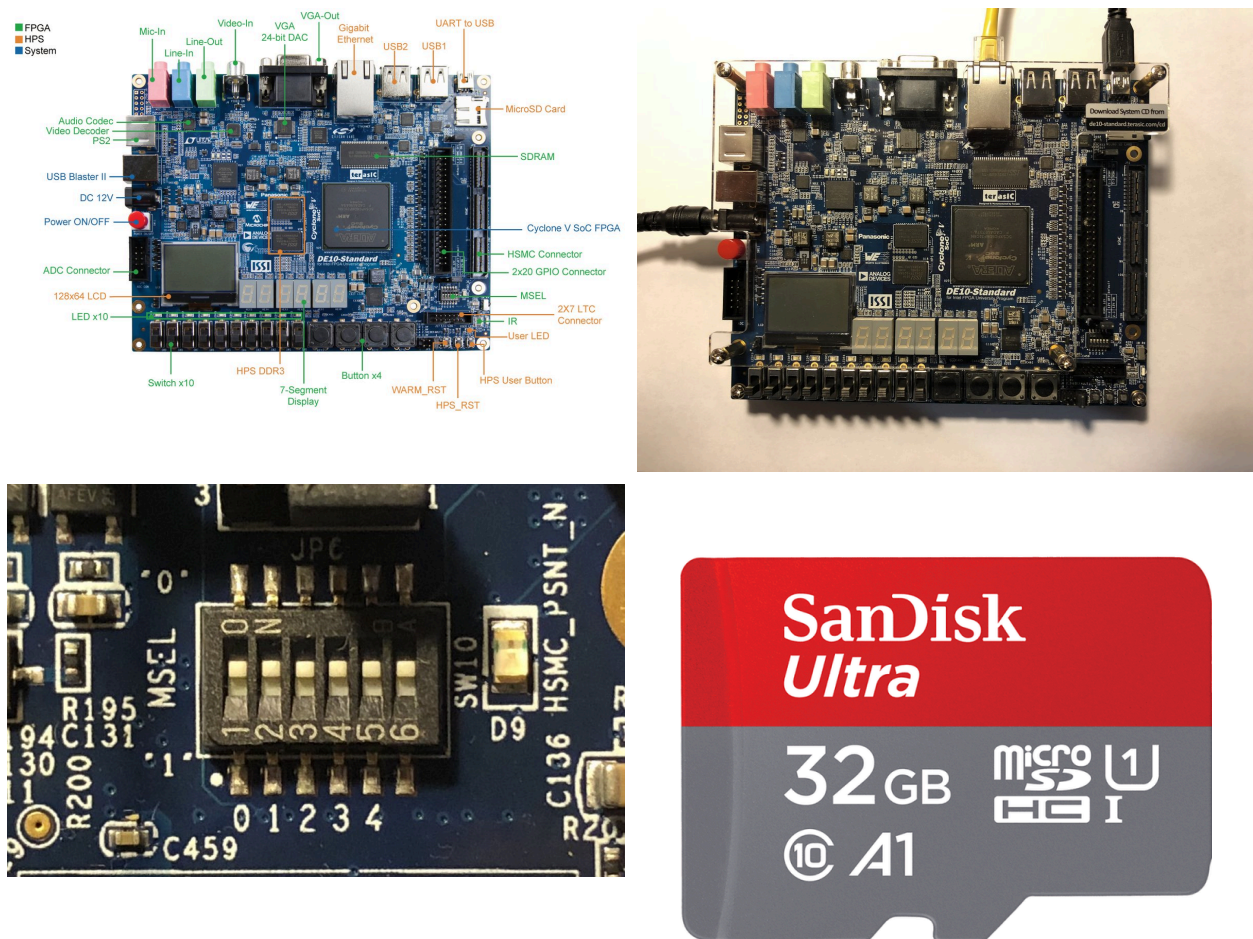
CHƯƠNG 2

Cơ sở hệ thống và định hướng thiết kế

Chương này tóm tắt các cơ sở kỹ thuật cần thiết để giải thích các quyết định thiết kế ở Chương 3. Mỗi mục được giữ ở mức tối thiểu, ưu tiên phân lý thuyết có liên hệ trực tiếp tới phần đã hiện thực.

2.1 Nền tảng Cyclone V SoC và DE10-Standard

Vì mạch **Cyclone V SoC** tích hợp hai vùng chức năng độc lập trên cùng một die: phần **HPS** dựa trên cụm ARM Cortex-A9 lõi kép và phần **FPGA fabric** mềm [2], [4]. Hai vùng này được kết nối qua các cầu **AXI bridge** hai chiều và chia sẻ chung bộ điều khiển DDR3. Trên DE10-Standard, công tắc **MSEL = 000000** được dùng để FPGA tự cấu hình từ HPS thông qua chuỗi **preloader** → **U-Boot** → **Linux** nạp từ MicroSD [3].



Hình 3 — Các thành phần nền của giai đoạn bring-up: board, sơ đồ kết nối, công tắc MSEL và thẻ MicroSD chứa preloader, U-Boot và kernel.

2.2 HPS Linux và Bridge HPS-FPGA

Hệ điều hành chạy trên HPS là bản phân phối Linux nhúng nhánh **SoCFPGA** của Yocto Project [10]. Sau khi U-Boot bàn giao, hạt nhân khởi tạo thiết bị HPS theo **device tree** (`socfpga.dtb`) và đăng nhập userland với `root / 1234`. Tập tiện ích userland do BusyBox cung cấp [11], trong đó có `ifconfig`, `udhcpc` cho mạng và đặc biệt là `devmem` để truy cập trực tiếp địa chỉ vật lý phục vụ điều khiển ngoại vi FPGA. Việc dùng HPS Linux thay cho bare-metal hoặc Nios II đem lại hai ưu thế: ngăn xếp TCP/IP đã sẵn trong nhân, và Python/shell có thể hiện thực dịch vụ ở mức cao mà không cần biên dịch chéo phức tạp.

Trao đổi giữa HPS và FPGA trong đề tài đi qua **Lightweight HPS-FPGA Bridge**, một cầu Avalon-MM 32 bit dành cho thanh ghi điều khiển có dải nhỏ và yêu cầu băng thông thấp; toàn bộ bridge được ánh xạ vào không gian địa chỉ HPS bắt đầu tại `0xFF200000` [2], [12]. Trong cấu hình của đề tài, sáu IP `pio_hex0 ... pio_hex5` được khai báo lần lượt với offset `0x40, 0x50, 0x60, 0x70, 0x80, 0x90`. Mỗi IP là một thanh ghi 32 bit chỉ ghi

(output-only), phần mềm có thể xem mỗi PIO như một biến `volatile uint32_t` ở địa chỉ vật lý cố định, với 7 bit thấp được FPGA tách thành tín hiệu điều khiển bảy đoạn `a–g` của một LED 7 đoạn theo cực tính active-low.

2.3 Ethernet và giao thức TCP

DE10-Standard sử dụng **EMAC1** trong HPS, kết nối tới PHY Gigabit qua giao tiếp **RGMII** (`HPS_ENET_GTX_CLK`, `TX_DATA[3:0]`, `RX_DATA[3:0]`, `MDIO`, `MDC` ...) — đều được khai báo ở top-level VHDL. Trên Linux, EMAC1 xuất hiện dưới tên `eth0`; nhóm dùng `udhcpc` xin IP qua DHCP [13].

Transmission Control Protocol (RFC 9293 [8]) cung cấp kênh truyền có hướng kết nối, giữ thứ tự byte và đảm bảo tin cậy — đặc biệt phù hợp với điều khiển có phản hồi: client mở kết nối, gửi lệnh ngắt, nhận phản hồi tường minh và đóng kết nối. Cả hai phía của đề tài dùng ngăn xếp socket theo mô hình của Stevens [9] và Tanenbaum [14]. Phía board, server tạo socket `AF_INET/SOCK_STREAM`, đặt `SO_REUSEADDR`, gọi `bind()` tới `0.0.0.0:5000`, `listen()` rồi vào vòng `accept()`. Phía client, ứng dụng tạo socket cùng họ, đặt timeout 5 s, `connect()` rồi `sendall()` / `recv()` rồi đóng kết nối.

2.4 Hiện thị 7 đoạn và mã hóa active-low

Cụm HEX của DE10-Standard gồm sáu LED 7 đoạn dạng **common anode**: bảy chân đoạn nối chung tới nguồn dương, một đoạn được kích hoạt khi ngõ vào của nó bị kéo xuống mức thấp. Do đó mỗi giá trị 7 bit ghi vào PIO HEX phản ánh trạng thái **bị tắt** của các đoạn `a–g`: bit `0` = đoạn được bật, bit `1` = đoạn tắt [3]. Quy ước này dẫn đến `'0' = 0b1000000 = 64`, `'8' = 0b0000000 = 0` và `' ' = 0b1111111 = 127`. Tập ký tự được hỗ trợ là tập con thuận lợi để mô phỏng chữ in hoa: `0123456789ABCDEFHLOPUY-<space>`; ký tự ngoài tập đều thay bằng khoảng trắng, giữ tổng độ dài hiển thị. Vì `HEX5` nằm trái còn `HEX0` nằm phải, để chuỗi đọc tự nhiên, ký tự thứ nhất phải được mã hóa và ghi xuống `HEX5`, ký tự cuối xuống `HEX0`.

2.5 Công cụ phát triển

P phía phần cứng và embedded, nhóm dùng **Intel Quartus Prime 18.1 Standard Edition** và **Platform Designer** để dựng hệ thống SoC và sinh `socfpga.rbf` [15]. Phía ứng dụng, GUI PC viết bằng Python với thư viện chuẩn `socket` và `tkinter` [16], [17]; Android client dùng API Networking trong Android Framework [18]. Báo cáo được biên soạn bằng Typst, trích dẫn theo chuẩn IEEE và quản lý tham chiếu qua BibTeX.

CHƯƠNG 3

Thiết kế và triển khai hệ thống

Chương này mô tả hệ thống do nhóm hiện thực, đi từ phần cứng tới phần mềm ứng dụng, theo trục **PC** → **TCP** → **HPS** → **Bridge** → **HEX**.

3.1 Tổng quan kiến trúc

Hệ thống tổ chức thành bốn lớp tách biệt, mỗi lớp có thể kiểm thử độc lập: **(i)** lớp ngoại vi FPGA — sáu PIO 7 bit `pio_hex0` – `pio_hex5` xuất tín hiệu trực tiếp tới LED 7 đoạn vật lý; **(ii)** lớp truy cập phần cứng — `devmem` từ user-space ghi giá trị 32 bit xuống các thanh ghi đã ánh xạ, hoặc script `hex_text.sh` đóng vai trò công cụ kiểm chứng độc lập; **(iii)** lớp dịch vụ — chương trình `board_tcp_hex_server.py` lắng nghe socket TCP, chuẩn hóa payload và sinh các lệnh ghi tới sáu địa chỉ PIO; **(iv)** lớp ứng dụng đầu cuối — GUI Python/Tkinter và Android client cùng thực hiện một giao thức yêu cầu/phản hồi đơn giản.

3.2 Thiết kế phần cứng

Hệ thống Platform Designer (`soc_system.qsys`) được mở rộng từ boilerplate SoC-FPGA của Kashani [5]. Các IP chính gồm: HPS với đầy đủ EMAC1/DDR3/SDIO/UART/PC/SPI/USB, PLL clock 50/100 MHz, một `pio_leds` rộng 10 bit cho `LEDR`, và **sáu PIO HEX** mỗi IP rộng 7 bit, output-only, nối tới Lightweight HPS-FPGA Bridge. Sau khi sinh HDL, hệ thống xuất `soc_system.vhd` cùng `hps_soc_system.h` chứa các hằng số địa chỉ thanh ghi.

Bản đồ thanh ghi PIO HEX trên Lightweight Bridge

Gốc bridge: `0xFF200000`. Sáu thanh ghi cách đều bước `0x10` :

<code>pio_hex0</code> → <code>0xFF200040</code> → <code>HEX0</code>	<code>pio_hex3</code> → <code>0xFF200070</code> → <code>HEX3</code>
<code>pio_hex1</code> → <code>0xFF200050</code> → <code>HEX1</code>	<code>pio_hex4</code> → <code>0xFF200080</code> → <code>HEX4</code>
<code>pio_hex2</code> → <code>0xFF200060</code> → <code>HEX2</code>	<code>pio_hex5</code> → <code>0xFF200090</code> → <code>HEX5</code>

Hình 4 — Bản đồ địa chỉ được dùng nhất quán giữa Platform Designer, `hex_text.sh` và `board_tcp_hex_server.py`.

Tệp `hw/hdl/project.vhd` khai báo entity `project` với toàn bộ chân I/O của DE10-Standard theo Terasic System Builder và khởi tạo component `soc_system`, đấu các port `pio_hex0_external_export` ... `pio_hex5_external_export` ra `HEX0..HEX5` vật lý. Đây là bước rất quyết định: nếu một chân HEX bị quên hoặc sai chiều, mọi tầng phần mềm phía trên đều không thể bù đắp.

3.3 Triển khai phần mềm trên HPS

Khởi động và bring-up mạng. Sau khi nạp `.sof`, board boot Linux từ MicroSD; nhóm mở serial console qua USB-UART (`/dev/ttyUSB0`, baud `115200`), đăng nhập `root/1234`, kích hoạt `eth0` và xin IP qua DHCP bằng `ifconfig eth0 up && udhcpc -i eth0` [13], đọc IP đã cấp và ghi nhận để cấu hình client.

Dịch vụ TCP `board_tcp_hex_server.py`. Đây là thành phần trung tâm phía board, tổ chức thành bốn hàm: `seg(ch)` ánh xạ ký tự sang mã 7 đoạn, `sanitize_text(s)` chuẩn hóa chuỗi, `write_hex(text)` sinh sáu lệnh `devmem`, và `handle_client(conn, addr)` đảm nhiệm vòng đời một kết nối. Server bind tới `0.0.0.0:5000` với `SO_REUSEADDR`, đóng kết nối sau mỗi yêu cầu (không giữ trạng thái phiên), và phản hồi văn bản `OK:<text>` hoặc `ERR:<lý do>`. Một quyết định cần làm rõ: server **không** gọi `hex_text.sh`. Bảng mã và quy tắc đảo thứ tự ghi được đưa thẳng vào Python để giảm số lần `fork/exec`; script `hex_text.sh` được giữ lại như công cụ kiểm chứng độc lập trong các phiên bring-up (Phụ lục B).

Lớp ánh xạ ký tự sang mã 7 đoạn. Bảng mã được xây dựng dựa trên quy ước active-low của board (mục 2.4), trích chính như sau:

Bảng mã <code>seg()</code> thực dùng (giá trị 7 bit, active-low)	
Số:	<code>0</code> → 64, <code>1</code> → 121, <code>2</code> → 36, <code>3</code> → 48, <code>4</code> → 25, <code>5</code> → 18, <code>6</code> → 2, <code>7</code> → 120, <code>8</code> → 0, <code>9</code> → 16.
Chữ:	<code>A</code> → 8, <code>B</code> → 3, <code>C</code> → 70, <code>D</code> → 33, <code>E</code> → 6, <code>F</code> → 14, <code>H</code> → 9, <code>L</code> → 71, <code>O</code> → 64, <code>P</code> → 12, <code>U</code> → 65, <code>Y</code> → 17.
Ký hiệu:	<code>-</code> → 63, <code>_</code> → 119, khoảng trắng → 127. Mọi ký tự không có trong bảng đều ánh xạ về 127 (LED tắt).

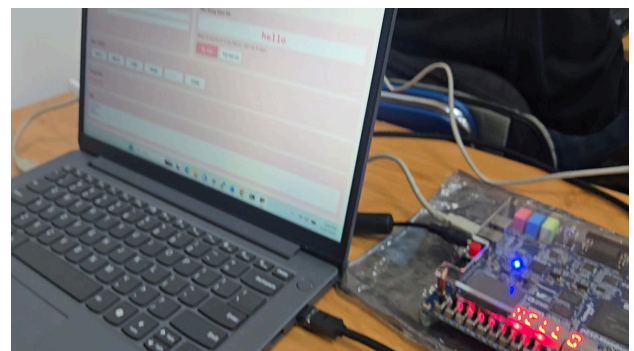
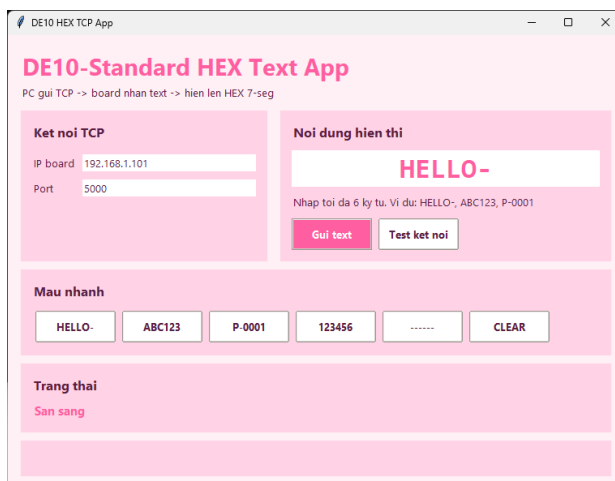
Hình 5 — Mã 7 đoạn của tập ký tự được hỗ trợ.

Quy tắc thứ tự ghi. Để chuỗi gửi từ PC hiển thị tự nhiên từ trái qua phải, server đảo thứ tự ghi: ký tự thứ nhất (chỉ số 0) được mã hóa và ghi xuống HEX5 (0xFF200090), ký tự cuối ghi xuống HEX0. Hàm `write_hex` hiện thực rõ:

```
1 v5, v4, v3, v2, v1, v0 = (seg(c) for c in text[:6])
2 for addr, val in zip(["0xFF200040", "0xFF200050", "0xFF200060",
3                      "0xFF200070", "0xFF200080", "0xFF200090"],
4                      (v0, v1, v2, v3, v4, v5)):
5     subprocess.check_call(["devmem", addr, "32", str(val)])
```

3.4 Ứng dụng đầu cuối

GUI PC `pc_hex_tcp_pink_gui.py`. Ứng dụng tổ chức theo lớp `App(tk.Tk)` với hai khu vực: khối cấu hình IP/cổng và payload, khối **mẫu nhanh** (HELLO-, ABC123, P-0001, 123456, -----,) cùng khung **log** phản hồi [17]. Hàm `tcp_send(text)` chứa toàn bộ logic giao tiếp: tạo socket, đặt timeout 5 s, `connect()`, `sendall((text + "\n").encode("utf-8"))`, `recv(1024)` rồi đóng [16]. Tác vụ TCP được chuyển sang `threading.Thread(daemon=True)` riêng để GUI không bị treo khi đợi phản hồi. Phía client cũng cắt payload ở sáu ký tự (`text_to_send.get()[:6]`) — kiểm tra dư so với phía server.



Hình 6 — Giao diện ứng dụng PC và tình huống vận hành thực tế.

Android client. Dựng tối giản với ô nhập IP/cổng/chuỗi và nút **Gửi**; logic gửi đặt trong background thread để không vi phạm `NetworkOnMainThreadException` [18]. Mục tiêu của Android client không phải thay thế PC GUI mà chứng minh dịch vụ board-side là **một**

dịch vụ TCP độc lập với đầu cuối: chỉ cần cùng mạng và đúng IP/cổng, một thiết bị di động cũng điều khiển được HEX với hành vi tương đương.

3.5 Giao thức ứng dụng

Trên nền TCP thuần, đề tài tự định nghĩa một giao thức ứng dụng tối thiểu: **yêu cầu** là chuỗi UTF-8 kết thúc bởi `\n` (vài chục byte); **phản hồi** là `OK:<text>\n` khi thành công, `ERR:<lý do>\n` khi lỗi; server đóng socket sau mỗi yêu cầu. Quy ước này tạo thuận lợi cho khoảng vùng lỗi: có `OK` mà HEX sai → lỗi ở bảng mã, thứ tự ghi hoặc top-level; có `ERR` và HEX không đổi → lỗi ở `devmem` hoặc bridge; không có cả `OK` lẫn `ERR` → lỗi xảy ra trước khi server kịp ghi (kết nối bị reset, server không chạy, IP/cổng sai).

CHƯƠNG 4

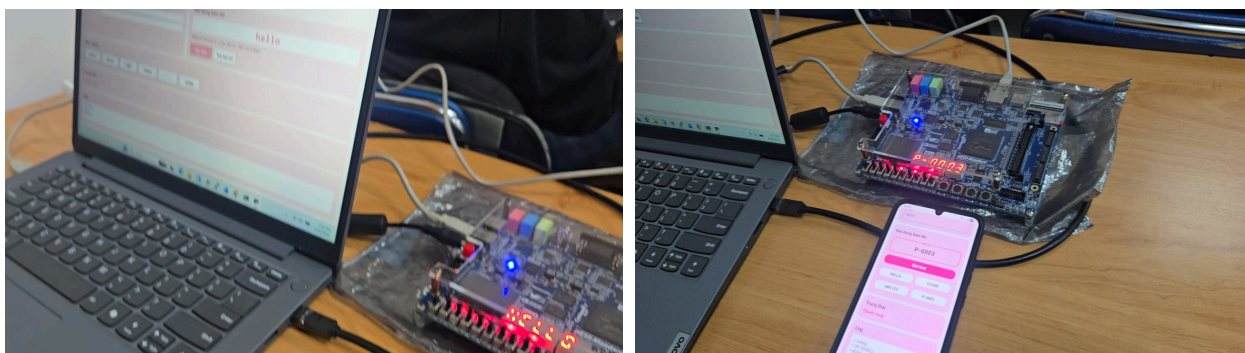
Kiểm thử và đánh giá hệ thống

Chương này trình bày phương pháp, môi trường và kết quả kiểm thử. Kiểm thử tập trung vào **tính đúng chức năng** và **khả năng lặp lại**; báo cáo chưa đặt mục tiêu đo các chỉ số định lượng (độ trễ, throughput, tải CPU) vì điều kiện đo lường chuẩn xác vượt khỏi phạm vi học phần.

4.1 Tiêu chí và môi trường

Hệ thống được xem là đạt khi đồng thời thỏa: HPS boot Linux và vào shell qua USB-UART; `eth0` có IP hợp lệ cùng dải với máy điều khiển; server TCP lắng nghe `5000` và xuất ra log khởi động; mỗi yêu cầu hợp lệ nhận về `OK:<text>` đúng định dạng; cụm `HEX0...HEX5` hiển thị đúng chuỗi đã chuẩn hóa, đọc tự nhiên từ trái qua phải; Android client cho hành vi tương đương khi cùng mạng.

Môi trường gồm board **DE10-Standard** gắn MicroSD, cáp Mini-USB cho serial console, cáp Ethernet nối tới router; **laptop điều khiển** chạy Windows 11 với Python 3.11+ chạy `pc_hex_tcp_pink_gui.py`; **điện thoại Android** chạy Android client trên Wi-Fi của cùng router. Việc đổi giữa PC và Android giữ nguyên board, server và mạng — chỉ thay thiết bị gửi lệnh — qua đó kiểm chứng dịch vụ board-side không phụ thuộc vào loại đầu cuối.



Hình 7 — Môi trường kiểm thử thực tế: laptop, điện thoại và board kết nối qua cùng router; mọi đầu cuối đều có thể nhìn thấy `eth0` của board.

4.2 Kịch bản và kết quả

Bộ kịch bản được thiết kế **phân tầng từ thấp lên cao**, mỗi bước chỉ chuyển tiếp khi bước trước đã chắc đúng:

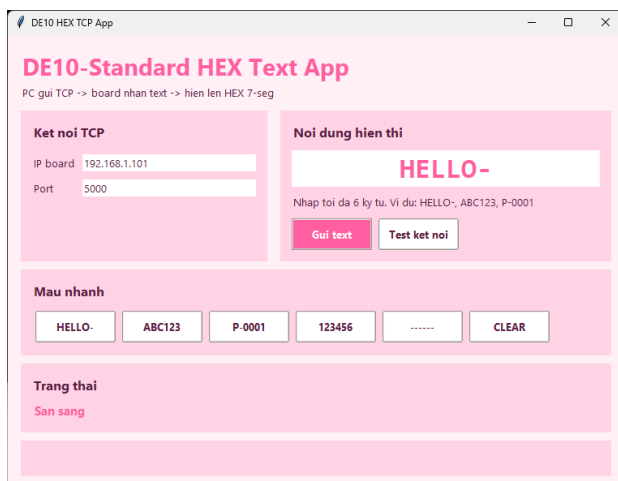
Năm bước kiểm thử

1. **Boot và shell:** quan sát log Linux trên USB-UART; xác nhận đăng nhập, không có lỗi panic, các module USB/SD/Ethernet được nhận diện.
2. **Mạng eth0:** gọi `ifconfig eth0 up && udhcpc -i eth0`, đợi DHCP cấp IP, ping board từ máy điều khiển.
3. **Dịch vụ TCP:** khởi động server, kiểm tra bằng `nc <ip-board> 5000` từ laptop để gửi chuỗi rỗng và quan sát log.
4. **Phản hồi yêu cầu:** gửi từng payload qua PC GUI, ghi nhận chuỗi `OK:<text>` trong khung log.
5. **Hiển thị:** đối chiếu giá trị trên `HEX5..HEX0` với chuỗi đã chuẩn hóa; chụp ảnh làm hiện vật.

Tập payload kiểm thử gồm `HELLO-`, `ABC123`, `P-0001`, `123456`, `-----` và sáu khoảng trắng để xóa hiển thị. Trường hợp biên: chuỗi rỗng `"`, chuỗi dài `HELLOWORLD`, chữ thường `hello-`, ký tự không hỗ trợ `MAX@!`.

Trên đầu cuối PC, mọi payload nằm trong tập hỗ trợ đều được phản hồi `OK:<text-da-chuan-hoa>` trong vài chục mili-giây và HEX hiển thị đúng. Các trường hợp biên: chuỗi rỗng cho sáu PIO HEX nhận `127` và toàn bộ LED tắt; `HELLOWORLD` được cả GUI và server cắt còn `HELLOW`; `hello-` được `sanitize_text` chuyển thành `HELLO-`; `MAX@!` cho hiển thị `_A__` (các ký tự ngoài bảng được thay bằng khoảng trắng).

Khung log của GUI tách biệt rất rõ ba loại sự kiện: **gửi đi** (dòng bắt đầu `>`), **nhận về** (dòng bắt đầu `<`) và **lỗi** (dòng bắt đầu `LOI:`). Trên Android, kết quả tương đương khi điện thoại nằm cùng subnet với board; nếu khác subnet (ví dụ Wi-Fi mạng khách), `connect()` thất bại sau timeout — phù hợp hành vi TCP khi không có đường tới đích. Việc đổi qua lại PC/Android trong cùng phiên server không gây bất thường vì server không lưu trạng thái phiên.



Hình 8 — Kết quả với PC GUI (trái) và Android client (phải) trên cùng dịch vụ TCP.

4.3 Phân tích lỗi và đánh giá

Một số tình huống lỗi điển hình đã gặp và xử lý: **sai IP board** sau reboot router cập lại IP — đọc `ifconfig eth0 | grep inet` rồi cập nhật GUI; **server không chạy** sau reboot Linux — chạy lại `python /tmp/board_tcp_hex_server.py`; **HEX không đổi mà vẫn OK** — quên nạp `.sof` mới sau khi sửa Platform Designer; **Android không kết nối** — điện thoại đăng nhập Wi-Fi mạng khách, chuyển về cùng SSID với laptop.

Cây khoanh vùng lỗi theo tầng

Không có shell → kiểm nguồn, MSEL `000000`, MicroSD, USB-UART driver.

Có shell, không IP → kiểm cáp Ethernet, đèn link, DHCP của router, log `udhcpc`.

Có IP, client không kết nối → firewall PC, server đang chạy, cổng `5000`.

OK nhưng HEX sai → bảng `seg()`, thứ tự ghi `HEX5..HEX0`, top-level VHDL.

ERR → đọc thông điệp, kiểm `devmem`, quyền root, địa chỉ thanh ghi.

Hình 9 — Sơ đồ khoanh vùng lỗi xếp theo thứ tự lan truyền vấn đề.

Đối chiếu tiêu chí, hệ thống đạt toàn bộ tiêu chí chức năng đề ra. Toàn bộ luồng vận hành đầu cuối có thể tái lập ổn định: từ lúc bật nguồn board tới khi HEX hiển thị chuỗi đầu tiên, thời gian thao tác thủ công khoảng **hai đến ba phút** khi đã quen quy trình; mỗi yêu cầu sau đó tốn vài chục mili-giây tính bằng đồng hồ tường (chưa đo định lượng).

Hạn chế đã ghi nhận: server **chưa tự khởi động** cùng Linux nên cần thao tác thủ công sau reboot; **chưa có frame ứng dụng** (header độ dài, checksum, mã lệnh) — payload là plain text, khó mở rộng cho các loại nội dung khác; **chưa xác thực client** — bất kỳ thiết bị nào trong cùng LAN đều có thể gửi lệnh; **chưa có số liệu định lượng** về độ trễ, throughput

hoặc khả năng chạy liên tục dài hạn. Những hạn chế này không vi phạm tiêu chí chức năng đã đặt mà là cơ sở cho các hướng phát triển ở Chương 5.

CHƯƠNG 5

Kết luận và hướng phát triển

5.1 Kết quả và ý nghĩa

Đề tài SoC – Ethernet tích hợp hệ thống, firmware và ứng dụng nhận lệnh TCP từ PC đã được hiện thực và kiểm thử trọn vẹn trong phạm vi học phần. Hệ thống Platform Designer với HPS đầy đủ EMAC1/DDR3/SDIO/UART và cụm sáu PIO 7 bit cho HEX0 – HEX5 được tổng hợp sạch trên Quartus Prime; HPS chạy Linux nhúng từ MicroSD; dịch vụ TCP `board_tcp_hex_server.py` lắng nghe ở cổng 5000, chuẩn hóa chuỗi, mã hóa 7 đoạn (active-low) và ghi `devmem` xuống các thanh ghi tại `0xFF200040 – 0xFF200090` qua **Lightweight HPS-FPGA Bridge**; GUI PC Python/Tkinter giữ vai trò đầu cuối điều khiển chính, Android client dùng để đối chiếu hành vi với một loại đầu cuối khác.

Toàn bộ chuỗi vận hành — PC → TCP → HPS Linux → `devmem` → Bridge → HEX — có thể tái lập một cách ổn định, có hiện vật đối chiếu ở mọi tầng (log shell, log GUI, ảnh kiểm thử). Đề tài kết nối được nhiều mảng kiến thức của học phần thành một mạch logic duy nhất: Platform Designer cung cấp ngoại vi và bản đồ địa chỉ, Linux nhúng cung cấp tầng mạng và cơ chế truy cập phần cứng từ user-space, ngăn xếp socket TCP nối đầu cuối với dịch vụ, bảng mã 7 đoạn và quy tắc thứ tự ghi biến chuỗi văn bản thành ánh sáng vật lý. Ngoài giá trị kiến thức, đề tài còn rèn luyện cho nhóm phương pháp **bring-up có kỷ luật**: kiểm thử từ tầng thấp lên cao, không bỏ qua tầng nào, và luôn giữ cho mỗi tầng có một cách kiểm tra độc lập.

5.2 Hạn chế và hướng phát triển

Bản nguyên mẫu hiện tại còn một số hạn chế (đã nêu chi tiết ở Chương 4): server chưa tự khởi động cùng Linux; giao thức ứng dụng chỉ là plain text + OK / ERR, chưa có frame, mã lệnh, checksum, xác thực; `devmem` đủ cho phạm vi nguyên mẫu nhưng không phải phương án bền vững (quyền root, không khóa truy cập đồng thời); và chưa có số liệu định lượng về độ trễ, throughput hay khả năng chạy dài hạn.

Hướng mở rộng đề xuất theo độ ưu tiên

1. **Khởi động dịch vụ tự động**: viết unit `systemd` hoặc script init để server TCP nạp ngay khi Linux hoàn tất bring-up.

2. **Chuẩn hóa giao thức:** định nghĩa frame `<len><cmd><payload><crc>`, hỗ trợ các nhóm lệnh hiển thị, đọc trạng thái, nháy LED.
3. **Thay `devmem` bằng driver hoặc `mmap`:** viết module character device hoặc dùng `mmap` trên `/dev/mem` từ một process duy nhất, giảm chi phí `fork / exec` và kiểm soát truy cập đồng thời.
4. **Bổ sung xác thực:** token chia sẻ trước hoặc TLS, đặc biệt khi mở rộng ra ngoài LAN.
5. **Đo lường định lượng:** sử dụng `iperf3`, ghi nhận trung bình và độ lệch chuẩn của thời gian chuyển đi (round-trip time) ở các kích thước payload khác nhau.
6. **Cải tiến đầu cuối:** PC GUI lưu lịch sử IP, kiểm tra `ping` trước khi gửi, ghi log dạng tệp; Android hoàn thiện xử lý lỗi mạng và thêm danh sách mẫu nhanh.

Việc tập trung vào nhánh **TCP-first** không phải để giới hạn khả năng hệ thống mà để tạo được một hiện vật đầy đủ và có thể giải thích được. Trên nền hiện vật đó, các hướng phát triển trên đều có thể được triển khai một cách tiệm tiến, từ những bổ sung đơn giản như tự khởi động dịch vụ tới những thay đổi kiến trúc lớn hơn như viết driver kernel hoặc thêm tầng bảo mật.

TÀI LIỆU THAM KHẢO

- [1] “Hướng dẫn về Khóa luận tốt nghiệp và Đồ án (Seminar) tốt nghiệp”. Tháng Tám 2023. Truy cập: 4 Tháng Năm 2026. [Online]. Available at: <https://fetel.hcmus.edu.vn/quy-dinh-danh-cho-sinh-vien/>
- [2] “Cyclone V Hard Processor System Technical Reference Manual”. 2020. Truy cập: 4 Tháng Năm 2026. [Online]. Available at: <https://docs.altera.com/r/docs/683126/current>
- [3] “DE10-Standard User Manual”. 2017. Truy cập: 4 Tháng Năm 2026. [Online]. Available at: <https://www.terasic.com.tw/cgi-bin/page/archive.pl?CategoryNo=205&Language=English&No=1081&PartNo=4>
- [4] “Cyclone V Device Overview”. 2018. Truy cập: 4 Tháng Năm 2026. [Online]. Available at: <https://docs.altera.com/r/docs/683694/current>
- [5] S. Kashani, *SoC-FPGA Design Guide — DE1-SoC Edition*. EPFL — École Polytechnique Fédérale de Lausanne, 2017. Truy cập: 4 Tháng Năm 2026. [Online]. Available at: <https://github.com/sahandKashani/SoC-FPGA-Design-Guide>
- [6] “Tài liệu thực hành Hệ thống nhúng”. 2024. Truy cập: 4 Tháng Năm 2026. [Online]. Available at: https://drive.google.com/file/d/1Of78n54i0q6dKRQBaSb7xZcjGfrNpVfm/view?usp=drive_link
- [7] “Tài liệu thực hành Hệ thống nhúng SOC với Quartus Prime và Platform Designer”. 2024. Truy cập: 4 Tháng Năm 2026. [Online]. Available at: <https://drive.google.com/drive/folders/1uMNU3RWw1IrXVlxRNQeEtL0ro-QBOoA7?usp=sharing>
- [8] W. M. Eddy, “Transmission Control Protocol (TCP)”, RFC 9293, tháng 8 2022. doi: [10.17487/RFC9293](https://doi.org/10.17487/RFC9293).
- [9] W. R. Stevens, B. Fenner, và A. M. Rudoff, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 3rd a.b. Addison-Wesley Professional, 2004. [Online]. Available at: <https://books.google.com/books?id=ptSC4LpwGA0C>
- [10] “The Yocto Project Reference Manual”. 2024. Truy cập: 4 Tháng Năm 2026. [Online]. Available at: <https://docs.yoctoproject.org/>

- [11] E. Andersen và D. Vlasenko, “BusyBox — The Swiss Army Knife of Embedded Linux”. 2024. Truy cập: 4 Tháng Năm 2026. [Online]. Available at: <https://busybox.net/>
- [12] “Avalon Interface Specifications”. 2023. Truy cập: 4 Tháng Năm 2026. [Online]. Available at: <https://docs.altera.com/r/docs/683091/current>
- [13] R. Droms, “Dynamic Host Configuration Protocol”, RFC 2131, tháng 3 1997. doi: [10.17487/RFC2131](https://doi.org/10.17487/RFC2131).
- [14] A. S. Tanenbaum và D. J. Wetherall, *Computer Networks*, 5th a.b. Pearson, 2011. [Online]. Available at: <https://www.pearson.com/en-us/subject-catalog/p/computer-networks/P2000000003188/9780132126953>
- [15] “Intel Quartus Prime Standard Edition User Guide: Platform Designer”. 2018. Truy cập: 4 Tháng Năm 2026. [Online]. Available at: <https://docs.altera.com/r/docs/683364/18.1/intel-quartus-prime-standard-edition-user-guide-platform-designer/creating-a-system-with-platform-designer>
- [16] “socket — Low-level Networking Interface”. 2025. Truy cập: 4 Tháng Năm 2026. [Online]. Available at: <https://docs.python.org/3/library/socket.html>
- [17] “tkinter — Python Interface to Tcl/Tk”. 2025. Truy cập: 4 Tháng Năm 2026. [Online]. Available at: <https://docs.python.org/3/library/tkinter.html>
- [18] “Android Developer Documentation — Networking”. 2024. Truy cập: 4 Tháng Năm 2026. [Online]. Available at: <https://developer.android.com/training/basics/network-ops/connecting>

PHỤ LỤC

Phụ lục cung cấp các quy trình và đoạn mã nguồn cốt lõi của đề tài, đủ để người đọc tái lập hệ thống một cách độc lập sau khi đã có boilerplate SoC-FPGA tương ứng.

Phụ lục A. Quy trình vận hành nhanh

Tóm tắt thao tác từ lúc bật board đến lúc gửi lệnh thành công

1. Kiểm tra nguồn 5 V, công tắc **MSEL = 000000**, MicroSD đã được nạp ảnh hệ thống, cáp Mini-USB và cáp Ethernet.
2. Mở serial console (115200 8N1) trên cổng `/dev/ttyUSB0` (Linux) hoặc cổng COM tương ứng (Windows) để quan sát Linux boot.
3. Đăng nhập với `root / 1234` khi xuất hiện dấu nhắc đăng nhập.
4. Bật Ethernet và xin IP: `ifconfig eth0 up && udhcpc -i eth0`.
5. Đọc IP đã được cấp: `ifconfig eth0 | grep inet`; ghi lại địa chỉ.
6. Chép `board_tcp_hex_server.py` (Phụ lục C) lên `/tmp/` và cấp quyền chạy.
7. Khởi chạy server: `python /tmp/board_tcp_hex_server.py &`.
8. Mở `pc_hex_tcp_pink_gui.py` trên máy điều khiển; nhập IP và cổng `5000`; gõ chuỗi tối đa sáu ký tự, nhấn **Gửi text** và quan sát HEX.

Phụ lục B. Lệnh cấu hình mạng tối thiểu và script `hex_text.sh`

```
1 # Bring-up Ethernet và xin IP qua DHCP
2 ifconfig eth0 up
3 udhcpc -i eth0
4 ifconfig eth0
```

Script `hex_text.sh` (đặt trong `/sw/hps/`) đóng vai trò công cụ kiểm chứng độc lập: gọi trực tiếp từ shell để xác nhận đường ánh xạ HEX hoạt động trước khi tích hợp vào dịch vụ TCP.

```
1 #!/bin/sh
2 set -eu
3 BASE=0xFF200000
```

```

4  A0=$((BASE + 0x40)); A1=$((BASE + 0x50)); A2=$((BASE + 0x60))
5  A3=$((BASE + 0x70)); A4=$((BASE + 0x80)); A5=$((BASE + 0x90))
6  seg() {
7      case "$1" in
8          0) echo 64 ;; 1) echo 121 ;; 2) echo 36 ;; 3) echo 48 ;;
9          4) echo 25 ;; 5) echo 18 ;; 6) echo 2 ;; 7) echo 120 ;;
10         8) echo 0 ;; 9) echo 16 ;;
11         A) echo 8 ;; B) echo 3 ;; C) echo 70 ;; D) echo 33 ;;
12         E) echo 6 ;; F) echo 14 ;; H) echo 9 ;; L) echo 71 ;;
13         O) echo 64 ;; P) echo 12 ;; U) echo 65 ;; Y) echo 17 ;;
14         '-' ) echo 63 ;; '_' ) echo 119 ;; ' ' ) echo 127 ;;
15         *) echo 127 ;;
16     esac
17 }
18 text="${1:-      }"
19 text=$(printf '%-6.6s' "$text" | tr '[:lower:]' '[:upper:]')
20 C0=$(printf '%s' "$text" | cut -c1); C1=$(printf '%s' "$text" | cut -
c2)
21 C2=$(printf '%s' "$text" | cut -c3); C3=$(printf '%s' "$text" | cut -
c4)
22 C4=$(printf '%s' "$text" | cut -c5); C5=$(printf '%s' "$text" | cut -
c6)
23 # left-to-right text on HEX5..HEX0 => reverse order to addresses 0..5
24 V5=$(seg "$C0"); V4=$(seg "$C1"); V3=$(seg "$C2")
25 V2=$(seg "$C3"); V1=$(seg "$C4"); V0=$(seg "$C5")
26 for pair in "$A0:$V0" "$A1:$V1" "$A2:$V2" "$A3:$V3" "$A4:$V4" "$A5:
$V5"; do
27     addr=${pair%:*}; val=${pair##*:}
28     devmem "$addr" 32 "$val" >/dev/null
29 done
30 echo "OK text='$text'"

```

Phụ lục C. TCP server `board_tcp_hex_server.py` (rút gọn)

```

1  import socket, subprocess
2
3  HOST, PORT = "0.0.0.0", 5000
4  ADDRS = ["0xFF200040", "0xFF200050", "0xFF200060",
5           "0xFF200070", "0xFF200080", "0xFF200090"]
6  ALLOWED = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ-_ "
7
8  def seg(ch):
9      table = {
10         "0": 64, "1": 121, "2": 36, "3": 48, "4": 25, "5": 18,
11         "6": 2, "7": 120, "8": 0, "9": 16,
12         "A": 8, "B": 3, "C": 70, "D": 33, "E": 6, "F": 14,
13         "H": 9, "L": 71, "O": 64, "P": 12, "U": 65, "Y": 17,
14         "-": 63, "_": 119, " ": 127,
15     }
16     return table.get(ch, 127)
17
18 def sanitize_text(s):
19     s = s.strip().upper()
20     s = "".join(ch if ch in ALLOWED else " " for ch in s)
21     return s[:6].ljust(6)
22
23 def write_hex(text):
24     v5, v4, v3, v2, v1, v0 = (seg(c) for c in text[:6])
25     for addr, val in zip(ADDRS, (v0, v1, v2, v3, v4, v5)):
26         cmd = ["devmem", addr, "32", str(val)]
27         if subprocess.call(cmd) != 0:
28             raise RuntimeError("devmem fail: " + " ".join(cmd))
29
30 def handle_client(conn, addr):
31     try:
32         data = conn.recv(1024)

```

 Python


```

33         if not data:
34             return
35         if not isinstance(data, str):
36             data = data.decode("utf-8", "ignore")
37         text = sanitize_text(data)
38         write_hex(text)
39         conn.sendall(("OK:%s\n" % text).encode("utf-8"))
40     except Exception as e:
41         try:
42             conn.sendall(("ERR:%s\n" % e).encode("utf-8"))
43         except Exception:
44             pass
45     finally:
46         conn.close()
47
48 def main():
49     s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
50     s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
51     s.bind((HOST, PORT)); s.listen(5)
52     print("TCP HEX server listening on %s:%s" % (HOST, PORT))
53     while True:
54         conn, addr = s.accept()
55         handle_client(conn, addr)
56
57 if __name__ == "__main__":
58     main()

```

Phụ lục D. Lỗi GUI PC `pc_hex_tcp_pink_gui.py` (rút gọn)

```

1 import socket, threading, tkinter as tk
2 from tkinter import ttk, messagebox
3
4 class App(tk.Tk):

```

 Python

```

5     def __init__(self):
6         super().__init__()
7         self.title("DE10 HEX TCP App")
8         self.ip = tk.StringVar(value="192.168.1.101")
9         self.port = tk.StringVar(value="5000")
10        self.text_to_send = tk.StringVar(value="HELLO-")
11        # ... build_ui() dựng các Frame, Entry, Button, Text log
12
13    def tcp_send(self, text):
14        ip = self.ip.get().strip()
15        port = int(self.port.get().strip())
16        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
17        s.settimeout(5)
18        s.connect((ip, port))
19        s.sendall((text + "\n").encode("utf-8"))
20        data = s.recv(1024).decode("utf-8", errors="replace")
21        s.close()
22        return data
23
24    def send_raw(self, text):
25        def worker():
26            try:
27                reply = self.tcp_send(text)
28                self.append("> " + text); self.append("< " +
29                    reply.strip())
30            except Exception as e:
31                self.append("LOI: " + str(e))
32                messagebox.showerror("TCP loi", str(e))
33        threading.Thread(target=worker, daemon=True).start()
34
35    def send_text(self):
36        self.send_raw(self.text_to_send.get()[:6])

```

```
37 if __name__ == "__main__":  
38     App().mainloop()
```

Phụ lục E. Đoạn nối top-level VHDL liên quan đến HEX

```
1  soc_system_inst : component soc_system port map (  
2      clk_clk          => CLOCK_50,  
3      reset_reset_n    => KEY(0),  
4      -- ... các ánh xạ HPS_DDR3_*, HPS_ENET_*, HPS_SD_*, HPS_UART_*  
      khác  
5      pio_leds_external_export => LEDR,  
6      pio_hex0_external_export => HEX0,  
7      pio_hex1_external_export => HEX1,  
8      pio_hex2_external_export => HEX2,  
9      pio_hex3_external_export => HEX3,  
10     pio_hex4_external_export => HEX4,  
11     pio_hex5_external_export => HEX5  
12 );
```

vhdl